Constraint Satisfaction Problem

AL-2002 Artifical Intelligence Spring 2026

Prepared By: Zill-E-Huma

🎯Constraint Satisfaction Problem

🔬Overview

The search algorithms we discussed so far (Bfs, Dfs, etc.) had no knowledge of the states representation (black box). That means it only have a knowledge of initial state, goal test, and how to generate next states.

Example: 8 puzzle problem

In a puzzle:

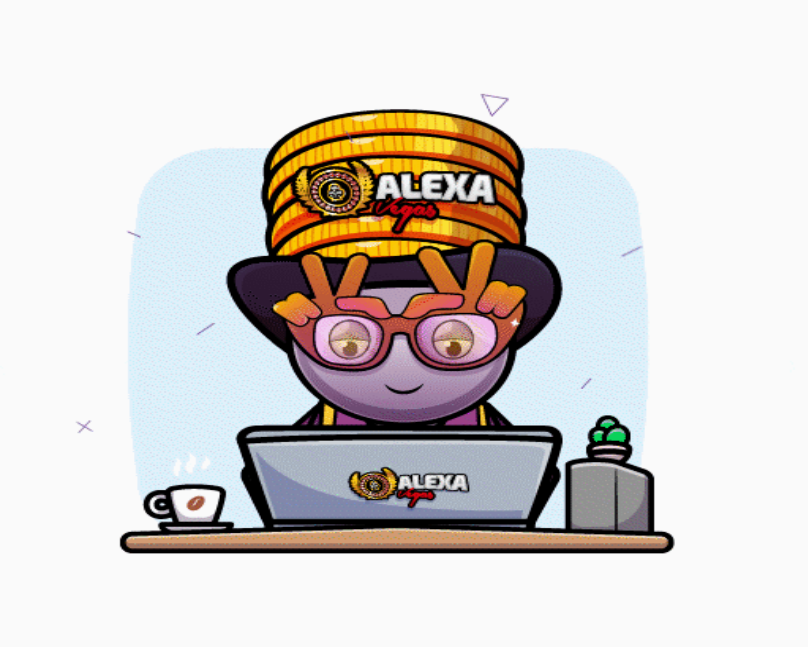
`State = board configuration`

Algorithm doesn't **"know"** it's a puzzle, it just sees some **data structure**.

– For each problem we had to design a new state representation.

- Instead we can have a general state representation that works well for many different problems.
- We can then build specialized search algorithms that operate efficiently on this general state representation.
- We call the class of problems that can be represented with this specialized representation.

Instead of treating problems as unknown **"black boxes,"** we define them in a common structured way (like CSP) so that specialized and more efficient algorithms can solve many problems without redesigning everything.



– Constraint Satisfaction Problems

CSP are mathematical questions defined as a set of objects, whose state must satisfy a number of constraints or limitations.

- CSP represent the entities in a problem as a homogeneous collection of finite constraints over variable, which is solved by constraint statisfaction method.
- The solution is typically a state that can satisfy all the constraint.

Components of Constraint Satisfaction Problems

Every CSP can be broken down into `three essential components`:

- Variables
- Domains
- Constraints.

1. **Variables:** These represent the elements that need to be assigned values. In a **Sudoku** problem, each cell in a grid considered as a variable.

   `V1 .... Vn`

   `Represented as position (row,column)`

   ```
   variables = [(i, j) for i in range(9) for j in range(9)]
   ```

2. **Domains:** Each variable has a domain, which is the set of possible values it can take. In the **Sudoku** example, the domain should be possible number {1 - 9} for each cell.

   `Dom[Vi]`

   `Empty cell -> {1 – 9}`

   `Already Filled cell -> Fixed value`

   ```
   domains = {
   var: set(range(1, 10)) if puzzle[var[0]][var[1]] == 0 else {puzzle[var[0]][var[1]]}
   for var in variables
   ```

   }

3. **Constraints**: Constraints define the rules that govern the relationships between variables. In **Sudoku** example, constraints dictate that no repitetion in a row, no repitetion in a column, and no repitetion in a 3 x 3 box.

   `C1 .... Cm`

   ```
   def add_constraint(var):
     constraints[var] = []
     for i in range(9):
         if i != var[0]:
             constraints[var].append((i, var[1]))
         if i != var[1]:
             constraints[var].append((var[0], i))
     sub_i, sub_j = var[0] // 3, var[1] // 3
     for i in range(sub_i * 3, (sub_i + 1) * 3):
         for j in range(sub_j * 3, (sub_j + 1) * 3):
             if (i, j) != var:
                 constraints[var].append((i, j))
   ```

Types of Constraint Satisfaction Problems

1. Binary CSPs

Binary CSPs are the simplest form of CSPs, where constraints exist between pairs of variables.

- Each constraint involves exactly two variables, making the problem easier to visualize and solve.
- An **example** of a binary CSP is the map coloring problem, where each region on a map must be assigned a color, and the constraint is that no two neighboring regions can share the same color. This problem can be represented as a graph, with nodes representing regions and edges representing constraints between neighboring regions.

```
# Variables (regions)
variables = ['A', 'B', 'C', 'D']

# Domains (colors)
domains = {
    'A': ['Red', 'Green', 'Blue'],
    'B': ['Red', 'Green', 'Blue'],
    'C': ['Red', 'Green', 'Blue'],
    'D': ['Red', 'Green', 'Blue']
}

# Constraints (neighbors: binary constraints)
constraints = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
```

```
}

# Check consistency (binary constraint: neighbors must differ)
def is_consistent(var, value, assignment):
    for neighbor in constraints[var]:
        if neighbor in assignment and assignment[neighbor] == value:
            return False
    return True
```

2. Non-binary CSPs

Non-binary CSPs involve constraints that apply to more than two variables.

- For **example**, in a scheduling problem, a constraint might specify that three tasks must be scheduled in different time slots.
- Non-binary CSPs are more complex than binary CSPs because they involve more intricate relationships between variables. Solving non-binary CSPs often requires breaking down the problem into smaller binary subproblems or using specialized algorithms that can handle higher-order constraints.

```
# Variables
variables = ['A', 'B', 'C']

# Domains (time slots)
domains = {
    'A': [1, 2, 3],
    'B': [1, 2, 3],
    'C': [1, 2, 3]
}

# Non-binary constraint: all variables must have different values
def all_different(assignment):
    values = list(assignment.values())
    return len(values) == len(set(values))  # no duplicates

# Check consistency (non-binary)
def is_consistent(assignment):
    return all_different(assignment)

# Rest CSP Algorithm
```

3. Dynamic CSPs

Dynamic CSPs are problems in which the variables or constraints can change over time. These problems are more flexible and require algorithms that can adapt to changes as they occur.

- A common **example** of a dynamic CSP is a real-time scheduling problem, where the availability of resources or the timing of tasks can change during the course of the problem-solving process. Dynamic CSPs are more challenging to solve because they require continuous updating and re-evaluation of the solution space.

CSP Algorithms

Constraint Satisfaction Problems (CSPs) are solved using various algorithms that improve the efficiency of the search process.

- These algorithms **`help narrow down the solution space`**, ensuring that constraints are satisfied while minimizing computational effort.
- Popular CSP algorithms include **`backtracking, forward-checking`**, and **`constraint propagation`**, each offering unique strategies for handling constraints effectively.

1. Backtracking Algorithm

The backtracking algorithm is a basic method for solving CSPs.

- It works by **assigning values** to variables `one by one`, checking at each step whether the constraints are satisfied. If a conflict is encountered, the algorithm backtracks and tries a different value.

```
def backtrack(self, assignment):
  if len(assignment) == len(self.variables):
      return assignment

  var = self.select_unassigned_variable(assignment)
```

```
    for value in self.order_domain_values(var, assignment):
        if self.is_consistent(var, value, assignment):

            # ------------------------
            # Step 1: Assign and Print
            # ------------------------
            assignment[var] = value
            print(f"Assign {value} at {var}")

            result = self.backtrack(assignment)
            if result is not None:
                return result

            # ------------------------
            # Step 2: Backtrack and Print
            # ------------------------
            del assignment[var]
            print(f"Backtrack at {var}")
    return None
```

- Backtracking is simple and effective for `small problems`, but it can be inefficient for large or complex CSPs because it explores every possible solution without any heuristic guidance.

2. Forward-Checking Algorithm

Forward-checking improves upon backtracking by reducing the search space. After each variable is assigned a value, forward-checking checks the remaining variables to ensure that there are still valid values available for them.

```
# Backtracking with forward checking
def backtrack(self, assignment, local_domains):
    if len(assignment) == len(self.variables):
        return assignment

    # Select unassigned variable (MRV)
    unassigned = [v for v in self.variables if v not in assignment]
    var = min(unassigned, key=lambda v: len(local_domains[v]))

    for value in local_domains[var]:
        if self.is_consistent(var, value, assignment):
            assignment[var] = value
            print(f"Assign {value} at {var}")

            # Forward check: reduce domains of neighbors
            temp_domains = {v: local_domains[v].copy() for v in local_domains}
            failure = False
            for neighbor in self.constraints[var]:
                if neighbor not in assignment and value in temp_domains[neighbor]:
                    temp_domains[neighbor].remove(value)
                    if len(temp_domains[neighbor]) == 0:
                        failure = True
                        break

            if not failure:
                result = self.backtrack(assignment, temp_domains)
                if result is not None:
                    return result

            # Backtrack
            print(f"Backtrack at {var}")
            del assignment[var]

    return None
```

- If a variable is found to have no valid values left, the algorithm backtracks immediately, avoiding unnecessary exploration of invalid solutions.
- This technique significantly reduces the number of solutions that need to be explored, making it more efficient than basic backtracking.

3. Constraint Propagation Algorithms

Constraint propagation algorithms, such as the Arc Consistency Algorithm (AC-3), enforce constraints during the search process by ensuring that each variable is consistent with the constraints before proceeding.

```
# ---------- Backtracking with AC-3 ----------
from collections import deque

def ac3(domains, constraints):
    queue = deque([(xi, xj) for xi in variables for xj in constraints[xi]])

    while queue:
        xi, xj = queue.popleft()
        if revise(xi, xj, domains):
            if len(domains[xi]) == 0:
                return False  # failure
            for xk in constraints[xi]:
                if xk != xj:
                    queue.append((xk, xi))
    return True



def revise(xi, xj, domains):
    revised = False
    for val in set(domains[xi]):
        if all(val == v for v in domains[xj]):
            domains[xi].remove(val)
            revised = True
            print(f"Remove {val} from {xi} because of {xj}")
    return revised
def backtrack(assignment, domains):
    if len(assignment) == len(variables):
        return assignment

    # Select unassigned variable (MRV)
    unassigned = [v for v in variables if v not in assignment]
    var = min(unassigned, key=lambda v: len(domains[v]))

    for value in domains[var]:
        assignment[var] = value
        print(f"Assign {value} at {var}")

        # Copy domains and assign value
        local_domains = {v: domains[v].copy() for v in domains}
        local_domains[var] = {value}

        # Apply AC-3 after assignment
        if ac3(local_domains, constraints):
            result = backtrack(assignment, local_domains)
            if result:
                return result

        print(f"Backtrack at {var}")
        del assignment[var]

    return None
```

- AC-3 works by iteratively checking the constraints between variables and removing inconsistent values from their domains.
- This process reduces the search space and eliminates values that cannot lead to valid solutions, making the algorithm more efficient at finding solutions.

Sudoku Problem Using CSP

```
# ------------------------------------------------------------
#                      8 Puzzle Board
# ------------------------------------------------------------
```

```
puzzle = [[5, 3, 0, 0, 7, 0, 0, 0, 0],
          [6, 0, 0, 1, 9, 5, 0, 0, 0],
          [0, 9, 8, 0, 0, 0, 0, 6, 0],
          [8, 0, 0, 0, 6, 0, 0, 0, 3],
          [4, 0, 0, 8, 0, 3, 0, 0, 1],
          [7, 0, 0, 0, 2, 0, 0, 0, 6],
          [0, 6, 0, 0, 0, 0, 2, 8, 0],
          [0, 0, 0, 4, 1, 9, 0, 0, 5],
          [0, 0, 0, 0, 8, 0, 0, 7, 9]]

def print_sudoku(puzzle):
  #   Loop that iterate through each row
    for i in range(9):
        if i % 3 == 0 and i != 0:
            print("- - - - - - - - - - - - ")

     #   Loop that iterate through each column
        for j in range(9):
            if j % 3 == 0 and j != 0:
                print(" | ", end="")
            print(puzzle[i][j], end=" ")
        print()

print("Initial Sudoku Puzzle:\n")
print_sudoku(puzzle)

class CSP:
    def __init__(self, variables, domains, constraints):
        self.variables = variables                  # All Sudoku cells (coordinates (i,j))
        self.domains = domains                      # Possible values for each variable (1–9 for empty, fixed for filled)
        self.constraints = constraints              # Rules for which variables cannot have the same value
        self.solution = None                        # Final Solution

    def solve(self):
        assignment = {}                             # Dictionary representing the current state e.g. {variable: value}
        self.solution = self.backtrack(assignment)
        return self.solution

    def backtrack(self, assignment):
      # ------------------------------------------------------------
      #                     Base Case
      # If all variables are assigned, we have a complete solution
      # ------------------------------------------------------------
        if len(assignment) == len(self.variables):
            return assignment
# Picks a cell (i,j) that is not assigned yet.
        var = self.select_unassigned_variable(assignment)
# This return the possible numbers for the cell that don't conflict with the already assigned neighbors
        for value in self.order_domain_values(var, assignment):
# Checks the value in var does not break Sudoku rules
            if self.is_consistent(var, value, assignment):
# Assign the value to the cell
                assignment[var] = value
# Call backtrack recursively to solve the rest of the Sudoku.
                result = self.backtrack(assignment)
                if result is not None:                                      # Stop searching
# If none of the recursive calls returned a solution, undo the assignment
                    return result
# If all values for this variable fail, no solution possible along this path.
                del assignment[var]
        return None

    def select_unassigned_variable(self, assignment):
        unassigned_vars = [var for var in self.variables if var not in assignment]

        # Chooses the variable with the fewest possible values left.
        return min(unassigned_vars, key=lambda var: len(self.domains[var]))

    def order_domain_values(self, var, assignment):

       # Returns the possible values (domain) for the variable var
        return self.domains[var]

    def is_consistent(self, var, value, assignment):
      # Checks whether assigning value to var breaks any constraints
        for constraint_var in self.constraints[var]:
            if constraint_var in assignment and assignment[constraint_var] == value:
                return False
        return True
# ------------------------------------------------------------------------------
#                       Defining Variables, Domains and Constraints
```

```
#                       Defining variables, domains and constraints
# ------------------------------------------------------------------------------
variables = [(i, j) for i in range(9) for j in range(9)]

domains = {
    var: set(range(1, 10)) if puzzle[var[0]][var[1]] == 0 else {puzzle[var[0]][var[1]]}
    for var in variables
}

constraints = {}

def add_constraint(var):
    constraints[var] = []
    for i in range(9):
        if i != var[0]:
            constraints[var].append((i, var[1]))
        if i != var[1]:
            constraints[var].append((var[0], i))
    sub_i, sub_j = var[0] // 3, var[1] // 3
    for i in range(sub_i * 3, (sub_i + 1) * 3):
        for j in range(sub_j * 3, (sub_j + 1) * 3):
            if (i, j) != var:
                constraints[var].append((i, j))

for var in variables:
    add_constraint(var)
csp = CSP(variables, domains, constraints)
sol = csp.solve()

solution = [[0 for _ in range(9)] for _ in range(9)]
for (i, j), val in sol.items():
    solution[i][j] = val

print("\n******* Solution *******\n")
print_sudoku(solution)
```

```
Initial Sudoku Puzzle:

5 3 0  | 0 7 0  | 0 0 0
6 0 0  | 1 9 5  | 0 0 0
0 9 8  | 0 0 0  | 0 6 0
- - - - - - - - - - - -
8 0 0  | 0 6 0  | 0 0 3
4 0 0  | 8 0 3  | 0 0 1
7 0 0  | 0 2 0  | 0 0 6
- - - - - - - - - - - -
0 6 0  | 0 0 0  | 2 8 0
0 0 0  | 4 1 9  | 0 0 5
0 0 0  | 0 8 0  | 0 7 9

******* Solution *******

5 3 4  | 6 7 8  | 9 1 2
6 7 2  | 1 9 5  | 3 4 8
1 9 8  | 3 4 2  | 5 6 7
- - - - - - - - - - - -
8 5 9  | 7 6 1  | 4 2 3
4 2 6  | 8 5 3  | 7 9 1
7 1 3  | 9 2 4  | 8 5 6
- - - - - - - - - - - -
9 6 1  | 5 3 7  | 2 8 4
2 8 7  | 4 1 9  | 6 3 5
3 4 5  | 2 8 6  | 1 7 9
```

Hope you liked today's class! Z :)